

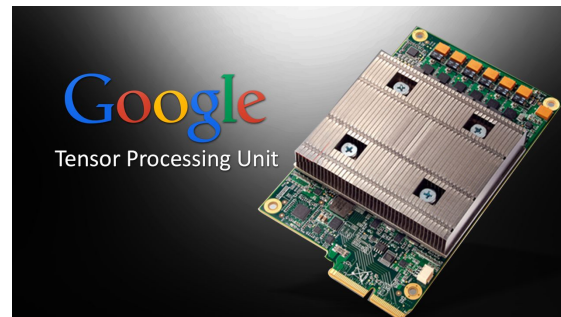
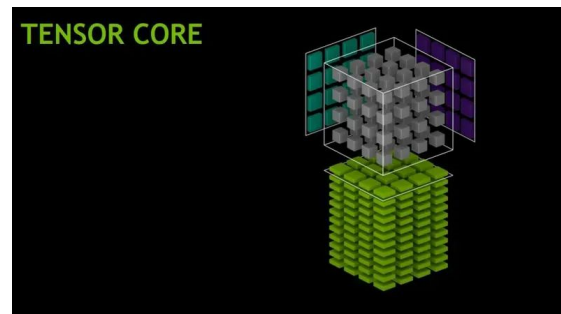
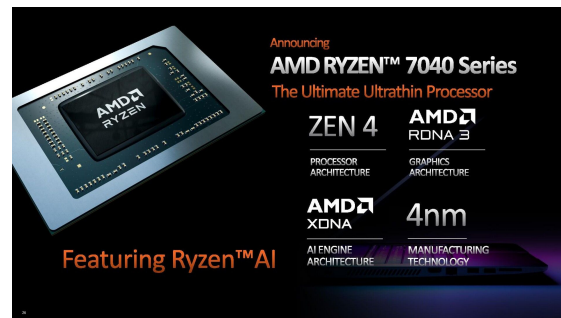
PhD Overview

**Tensor Program Optimization through
Program Synthesis**

Alexander Brauckmann

Context

- End of Moores law
- Specialized tensor hardware (GPUs, TPUs) give massive performance gains
- However difficult to program
- Domain-specific Languages (DSLs) allow managing programming complexity
- E.g. numeric domain: Tensor DSLs



Two Major Challenges

```
for (int i = 0; i < M; i++) {  
  for (int j = 0; j < N; j++) {  
    D[i][j] = C[i][j];  
    for (int k = 0; k < P; k++) {  
      D[i][j] += A[i][k] * B[k][j];  
    }  
  }  
}
```

The Legacy Code Gap

- Legacy codebases cannot access modern hardware or DSL compiler optimizations
- Existing methods - pattern matching, verified lifting - are brittle, manual, and fail to scale.

Program 1

diag(A @ B)

Algorithmic Replacement

sum(A * B.T, axis=1)

Program 2

$(\sqrt{x} + \sqrt{x})^2$

Algebraic Simplification

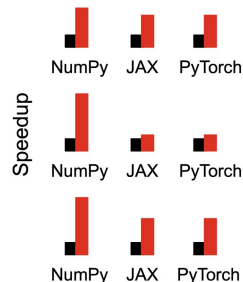
4 * x

Program 3

a * A @ B

Reordering Reductions

a * (A @ B)



The Optimization Gap

- State-of-the-art compilers like JAX and PyTorch Inductor miss powerful algorithmic and algebraic rewrites
- Existing methods - pattern matching, egraphs - are incomplete.

PhD: Program Synthesis for Tensor DSLs

MlirSynth: Profitability

Bottom-up enumerative synthesis confirms profitability.

21.6x speedups on TPUs, but exponentially scalable.

Tensorize: Scalable, Correct

Reformulated synthesis as top-down symbolic search.

Search guided by algebraic progress.

Achieves linear scaling in most benchmarks.

STENSO: Optimization

Exhaustive symbolic search, pruned by cost model and branch-and-bound.

Automatically discovers non-obvious rewrites, achieving 1.6x geo-mean speedup over JAX and PyTorch Inductor.

Legacy Code Gap

Optimization Gap

Limitations & Future Work

Floating-Point Correctness

Correctness guarantees rests on real-number algebra (SymPy), not IEEE 754 equivalence. Mirrors standard compiler trade-offs.

Future work on tunable precision guarantees.

Data-dependent Control Flow

Static control flow only. Data-dependent control flow bounds is not supported.

Future work on symbolic reasoning for unbounded structures.

Unbounded Correctness

Guaranteed correctness only up to certain data sizes.

Future work on composability of loop invariants.

Large Language Models (LLMs) as Guides

LLMs excels at candidate generation but lack formal correctness guarantees. Symbolic synthesis serves as the verification and repair engine, unlocking verified LLM-guided code generation.

Conclusion

- Program Synthesis is a **robust, correct, and scalable** method for solving several pressing challenges in tensor program optimization.
- **MlirSynth** and **Tensorize** automate legacy code lifting, effectively bridging the *legacy code gap*
- **STENSO** automates optimization by discovering better programs, effectively bridging the *optimization gap*